

Aula 3 — As rotas de autenticação

Duração: ~3h · **Você sai daqui com:** cadastro, ativação, login, logout e recuperação de senha.

Checkpoint: `git checkout aula-03`

1. O caminho de uma requisição

```
requisição → rota → controller → service → model → banco
                        ↓
                    serializer → JSON
```

Nesta aula todas essas peças aparecem. A regra que organiza tudo:

Controller recebe requisição e devolve resposta. Regra de negócio não mora nele.

2. A arquitetura de pastas

```
app/controllers/
├─ application_controller.rb
├─ concerns/api/v1/
│  ├─ authentication.rb      # quem está chamando?
│  └─ error_rendering.rb    # um formato de erro só
└─ api/v1/
   ├─ base_controller.rb    # pai de todos: erros, auth, strong params
   ├─ registrations_controller.rb
   ├─ email_verifications_controller.rb
   ├─ sessions_controller.rb
   ├─ password_resets_controller.rb
   └─ me_controller.rb

app/services/                # os casos de uso
├─ auth/
└─ users/

app/serializers/            # o que vai para o JSON
```

Por que service object

O `Users::Register` faz seis coisas: valida campos obrigatórios, aplica a política de senha, confere a confirmação, normaliza os dados, cria o usuário e dispara o e-mail. Dentro do controller isso vira um método de 60 linhas que só dá para testar subindo uma requisição HTTP inteira.

Fora dele:

```
result = Users::Register.call(name: "...", email: "...", ...)
result.success? # true/false
result.user
result.errors # [{ field: "password", message: "..."}]
```

Testável direto, reaproveitável (uma task de importação chama o mesmo service) e o controller volta a ter cinco linhas. O padrão sempre igual: **um call de classe, um Result**.

Por que serializer

```
render json: user
```

Isso manda o objeto inteiro: `password_digest`, `email_verification_code_digest`, `password_reset_code_digest`. Vazamento em uma linha.

O serializer é a lista de convidados: só entra quem está escrito lá.

```
class UserSerializer
  def self.as_json(user)
    { id: user.id, name: user.name, email: user.email, ... }
  end
end
```

Um formato de erro só

```
{
  "error": {
    "code": "validation_failed",
    "message": "Falha na validação",
    "details": [{ "field": "password", "message": "deve incluir pelo menos um dígito" }]
  }
}
```

- **code** é estável — o cliente decide o que fazer com base nele.
- **message** é para o usuário ler. Pode mudar, pode ser traduzida.
- **details** diz qual campo falhou, para o app pintar o input de vermelho.

Uma API que devolve erro de três jeitos diferentes força o cliente a tratar os três.

`before_action`: o filtro

```
class MeController < BaseController
  before_action :authenticate_user!

  def show
    render_user(current_user) # só roda se passou pelo filtro
  end
end
```

O filtro roda **antes** da action. Se ele renderizar alguma coisa — o `401` — a action nem chega a ser chamada. `only:` e `except:` limitam a quais actions ele se aplica:

```
before_action :authenticate_user!, only: :destroy
```

A pilha de middleware

A requisição não cai direto no controller. Ela atravessa uma pilha de camadas — o **middleware**. Cada camada pode ler, alterar, ou responder e cortar o caminho ali mesmo.

```
requisição
  ↓
[ Rack::Cors ]           ← libera (ou não) a origem
[ ActionDispatch::Cookies ]
[ ... ]
  ↓
rota → before_action → controller → service → model
  ↓
serializer → JSON → resposta (voltando pela mesma pilha)
```

```
bin/rails middleware # lista a pilha inteira do seu app
```

O modo `--api` remove várias camadas. Trouxemos os cookies de volta na mão, em `config/application.rb`:

```
config.middleware.use ActionDispatch::Cookies
```

Strong parameters

```
def registration_params
  expect_root_params(:name, :email, :password, :confirm_password,
                    :course, :matricula, :check_terms_use)
end
```

`params.expect` (Rails 8) **exige** essas chaves e **recusa o resto**. Sem isso alguém manda `"role": "admin"` no cadastro e vira admin. O nome disso é **mass assignment**, e já derrubou sistema grande.

3. Rota 1 — Cadastro

POST /api/v1/registrations

```
{
  "name": "Diego Reis",
  "email": "diego@aluno.ufop.edu.br",
  "password": "Automic@2026",
  "confirm_password": "Automic@2026",
  "course": "Engenharia de Minas",
  "matricula": "2013333",
  "check_terms_use": true
}
```

→ `201 Created` com o usuário e um e-mail de ativação a caminho.

A decisão que não é óbvia

```
REGISTRATION_FAILED_MESSAGE =
  "Não foi possível concluir o cadastro. Verifique os dados e tente novamente."
```

Quando o e-mail já existe, a resposta **não** diz "este e-mail já está cadastrado". Se dissesse, o cadastro viraria um verificador: eu testo mil e-mails e descubro quem tem conta no sistema. Chama-se **enumeração de usuários**, e vai voltar duas vezes nesta aula.

O custo é real: o usuário legítimo que esqueceu que já tem conta fica sem uma mensagem clara. A saída é o texto convidar a usar "esqueci minha senha".

4. E-mail: o mailer

Um mailer é um controller cujas views são e-mails.

```
class UserMailer < ApplicationMailer
  def email_verification(user, code)
    @user = user
    @code = code
    mail(to: @user.email, subject: "Ative sua conta")
  end
end
```

O template fica em `app/views/user_mailer/email_verification.text.erb`.

Em desenvolvimento não existe SMTP. O `letter_opener` abre o e-mail no navegador:

```
config.action_mailer.delivery_method = :letter_opener
```

`deliver_now` e não `deliver_later`

```
UserMailer.email_verification(user, code).deliver_now
```

O jeito "certo" de livro seria `deliver_later`, jogando numa fila. Aqui não, e o motivo é o código: numa fila, o código de 6 dígitos ficaria **gravado em texto** na tabela de jobs.

O preço é a requisição esperar o SMTP. Por isso o service tem `rescue`: falha de envio não pode virar 500.

```
rescue StandardError => e
  Rails.error.report(e, handled: true, source: "SendEmailVerification")
  Result.new(success?: false, error_code: "delivery_failed", ...)
end
```

5. Ativação da conta

```
POST /api/v1/email_verifications/confirm { email, code }
POST /api/v1/email_verifications/resend  { email }
```

O `resend` **sempre** responde 200 com a mesma mensagem — conta inexistente, já verificada ou em cooldown são indistinguíveis. É a mesma regra do cadastro.

6. O problema: HTTP não tem memória

Lembra da Aula 1? Cada requisição começa do zero. Então como o servidor sabe que você já entrou?

Duas famílias de resposta:

Sessão no servidor	Token
O servidor guarda a sessão e manda um id no cookie	O servidor manda um crachá assinado e não guarda nada
Toda requisição consulta o armazenamento de sessão	A assinatura basta: só validar
Logout é apagar a sessão — imediato	Logout é o problema difícil (já chegamos lá)
Escalar exige sessão compartilhada entre servidores	Qualquer servidor valida sozinho

Escolhemos **token**, porque o cliente principal é um app nativo — que não tem cookie e roda em milhares de celulares.

JWT

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJIsInZlciI6MH0 . 4pQ8L_5f...
cabeçalho          payload          assinatura
```

Três partes em **Base64**, separadas por ponto.

Base64 não é segredo. É uma forma de escrever bytes usando só letras e números, para caber num cabeçalho HTTP — que é texto. Não tem chave, não tem criptografia, qualquer um desfaz:

```
bash echo eyJzdWIiOiJ9 | base64 -d # {"sub":2}
```

Aquele monte de caractere embaralhado do JWT é isso: texto disfarçado.

O payload é público. Cole um token em jwt.io e você lê tudo. A assinatura garante que ninguém **alterou** o conteúdo, não que ninguém **leu**. Nunca ponha no payload nada que não possa ser lido — nem CPF, nem e-mail, nem permissão sensível.

O nosso payload:

```
{
  sub: user.id,           # subject: de quem é o token
  ver: user.token_version, # versão das sessões
  jti: SecureRandom.uuid, # id único deste token
  exp: 24.hours.from_now.to_i, # expiração
  iat: Time.current.to_i    # emitido em
}
```

O detalhe que quebra tudo

```
JWT.decode(token, secret, true, { algorithm: "HS256" })
#                               ^^^^
```

Esse `true` é a validação da assinatura. Com `false`, o `JWT.decode` aceita **qualquer** token — e qualquer pessoa forja o próprio acesso trocando o `sub`. Já foi CVE em várias bibliotecas.

O teste que prova isso está em `me_controller_test.rb`:

```
test "recusa token assinado com outro segredo" do
  forjado = JWT.encode({ sub: users(:ana).id, ... }, "segredo-do-atacante", "HS256")
  get "/api/v1/me", headers: auth_headers(forjado)
  assert_response :unauthorized
end
```

7. Rota 2 — Login

```
POST /api/v1/sessions { email, password, client }
```

`client` é `"mobile"` ou `"web"`, e decide **como** o token é entregue:

Cliente	Transporte	Por quê
<code>mobile</code>	cabeçalho <code>Authorization: Bearer <token></code>	App nativo não tem cookie
<code>web</code>	cookie assinado e <code>httpOnly</code>	JavaScript não lê o cookie, então um XSS não rouba o token

O que é um cookie

Um par `nome=valor` que o servidor manda no cabeçalho `Set-Cookie`. O navegador guarda e **reenvia sozinho**, em toda requisição àquele site. É a memória que o HTTP não tem, colada por fora.

Quem faz esse trabalho é o navegador — o app nativo não participa disso. Por isso o `client`.

```
cookies.signed[AUTH_COOKIE] = {  
  value: token,  
  httponly: true,           # JavaScript não enxerga  
  secure: Rails.env.production?, # só trafega em HTTPS  
  same_site: :lax          # mitiga CSRF  
}
```

`signed` significa que o Rails carimba o valor: alterou na mão, o Rails recusa.

XSS

Cross-Site Scripting: o atacante consegue rodar **JavaScript dentro da sua página**. Como? Você exibiu texto de usuário sem escapar — alguém salvou `<script>...</script>` como nome e a sua página imprimiu cru.

Com JavaScript rodando ali, ele lê tudo que o JavaScript lê. É exatamente por isso que `httpOnly` existe: o token no cookie fica **fora do alcance** do JavaScript, e um XSS não o rouba.

CSRF

Cross-Site Request Forgery. Lembra que o navegador reenvia o cookie sozinho?

Um site malicioso monta um formulário apontando para a sua API. A vítima clica; o navegador anexa o cookie dela; a sua API obedece, achando que foi ela quem pediu.

`same_site: :lax` manda o navegador **não enviar o cookie** quando a requisição parte de outro site.

API com token no cabeçalho não sofre de CSRF: ninguém anexa o `Authorization` por você. O problema é exclusivo de quem autentica por cookie — ou seja, do nosso `client: "web"`.

401 × 403

```
when "email_unverified"
  status: :forbidden      # 403: sabemos quem é você, mas ainda não pode
else
  status: :unauthorized   # 401: não sabemos quem é você
end
```

A mesma mensagem para os dois erros

E-mail inexistente e senha errada devolvem `invalid_credentials`, o mesmo código e a mesma mensagem. Terceira vez que a enumeração aparece.

E lembra do `DUMMY_PASSWORD_DIGEST` da Aula 2? Ele fecha o buraco pelo lado do **tempo**: não adianta a mensagem ser igual se a resposta volta em 1ms quando o e-mail não existe e em 100ms quando existe.

8. Rota 3 — Logout, o problema difícil

Um JWT é válido até expirar. O servidor não guarda sessão. **Então "sair" não existe** — o token continua funcionando por até 24h.

Três respostas possíveis, e nós usamos duas:

a) Apagar no cliente

O app joga o token fora. Resolve o caso normal e **não resolve nada** se alguém copiou o token.

b) Denylist por `jti` — revoga aquele token

```
module Auth::TokenDenylist
  def revoke(payload)
    jti = payload[:jti]
    ttl = payload[:exp].to_i - Time.current.to_i
    return false if jti.blank? || ttl <= 0

    Rails.cache.write(key(jti), true, expires_in: ttl.seconds)
  end

  def revoked?(jti)
    Rails.cache.exist?(key(jti))
  end
end
```

A sacada está no **TTL**: cada entrada vive exatamente o tempo que faltava para o token expirar. Depois disso o token morre sozinho e a entrada não serve mais. **A lista se limpa sem varredura e sem lixo acumulado.**

Isso revoga **um** token. Logout no celular não derruba o notebook — que é o comportamento certo.

c) `token_version` — derruba tudo de uma vez

```
def token_version_matches?(submitted_version)
  token_version == submitted_version.to_i
end

def invalidate_sessions!
  update!(token_version: token_version + 1)
end
```

O token carrega o `ver` de quando foi emitido. Incrementar a coluna no banco invalida **todos** os tokens do usuário, em todos os dispositivos. Usamos isso na troca de senha.

A verificação completa

```
def user_from_token(token)
  payload = Auth::DecodeToken.call(token) # 1. a assinatura confere?
  return if Auth::TokenDenylist.revoked?(payload[:jti]) # 2. foi revogado?

  user = User.find_by(id: payload[:sub])
  user if user&.token_version_matches?(payload[:ver]) # 3. a versão bate?
rescue Auth::DecodeToken::InvalidToken
  nil
end
```

9. Rota 4 — Recuperação de senha

Dois passos, e o mais interessante do dia.

```
POST /api/v1/password_resets/request { email }
POST /api/v1/password_resets/confirm { email, code, password, confirm_password }
```

`request` sempre responde 200

```
GENERIC_MESSAGE = "Se o e-mail estiver cadastrado, enviamos um código para redefinir sua senha."
```

Sempre. E-mail inexistente, não verificado, em cooldown, SMTP fora do ar — mesma resposta, mesmo status. Se respondesse 404 para e-mail inexistente, esta rota pública viraria um verificador de cadastro. É a quarta vez que o assunto aparece: **em fluxo de autenticação, respostas diferentes vazam informação.**

Duas guardas dentro:

- **Só conta verificada** recebe código. Senão dá para sequestrar um cadastro feito com o e-mail de outra pessoa antes de ela confirmar.
- **Cooldown de 1 minuto**, senão qualquer um usa o seu servidor de e-mail para incomodar terceiros.

`confirm` e a transação

```
User.transaction do
  user.save!           # troca a senha
  user.invalidate_sessions! # derruba todas as sessões ativas
  user.clear_password_reset_code! # consome o código
end
```

As três juntas ou nenhuma. Sem a transação, uma falha no meio deixa a senha trocada com o código ainda valendo.

E a ordem das validações importa: a política de senha é conferida **antes** de consumir o código. Senão o usuário que digita a senha nova errada perde o código e precisa pedir outro.

`invalidate_sessions!` **é o ponto do fluxo inteiro.** Se alguém invadiu a conta, trocar a senha tem que expulsar o invasor. Sem essa linha, ele continua logado com o token antigo.

10. Testando

```
bin/rails test
```

O teste que resume a aula:

```
test "logout revoga o token apresentado" do
  token = login_as(users(:ana))

  get "/api/v1/me", headers: auth_headers(token)
  assert_response :ok

  delete "/api/v1/sessions", headers: auth_headers(token)

  get "/api/v1/me", headers: auth_headers(token)
  assert_response :unauthorized # sem a denylist, isto daria 200
end
```

Pegadinha: em teste o `Rails.cache` padrão é o `null_store` — não guarda nada. O teste acima passaria "de graça", provando nada. Por isso o `test_helper.rb` troca por um `MemoryStore`.

As ferramentas de qualidade

```
bin/rubocop      # estilo – o black/ruff do Ruby
bin/brakeman     # análise estática de segurança
bundle exec bundler-audit check --update # gems com CVE conhecido
```

As três rodam no CI (Aula 4). O `brakeman` procura SQL injection, mass assignment, redirect aberto e mais uma dúzia de padrões.

11. CORS

```
origins(ENV.fetch("CORS_ORIGINS", "http://localhost:5173").split(","))
```

O navegador bloqueia, por padrão, uma página em `painel.exemplo.com` chamar `api.exemplo.com`. Esta config é a API dizendo quais origens aceita.

O app nativo não passa por CORS — isso é regra de navegador. Na prática, essa configuração existe por causa do painel web.

12. O fluxo inteiro, no terminal

```
API=localhost:3000/api/v1

# cadastro
curl -X POST $API/registrations -H 'Content-Type: application/json' -d '{
  "name":"Diego Reis","email":"diego@aluno.ufop.edu.br",
  "password":"Automic@2026","confirm_password":"Automic@2026",
  "course":"Engenharia de Minas","matricula":"2013333","check_terms_use":true}'

# login antes de confirmar → 403 email_unverified
curl -X POST $API/sessions -H 'Content-Type: application/json' \
  -d '{"email":"diego@aluno.ufop.edu.br","password":"Automic@2026","client":"mobile"}'

# pegue o código no e-mail que abriu no navegador, e confirme
curl -X POST $API/email_verifications/confirm -H 'Content-Type: application/json' \
  -d '{"email":"diego@aluno.ufop.edu.br","code":"123456"}'

# login (o token vem no cabeçalho Authorization da resposta)
curl -i -X POST $API/sessions -H 'Content-Type: application/json' \
  -d '{"email":"diego@aluno.ufop.edu.br","password":"Automic@2026","client":"mobile"}'

TOKEN=cole-o-token-aqui
curl $API/me -H "Authorization: Bearer $TOKEN"           # 200
curl -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl $API/me -H "Authorization: Bearer $TOKEN"           # 401
```

Exercício da aula

1. Monte as cinco rotas.
2. No **Insomnia**, crie uma coleção com uma requisição para cada uma.
3. Percorra o fluxo inteiro: cadastro → e-mail → confirmação → login → `/me` → logout → `/me` (401).
4. Peça recuperação de senha, troque a senha e confira que **o token antigo parou de funcionar**.
5. Cole o seu token em jwt.io e leia o payload. Ache o `sub`, o `jti` e o `exp`.
6. Rode `bin/rails test`, `bin/rubocop` e `bin/brakeman`.

Bônus 1: crie `PATCH /api/v1/me` para o usuário editar o próprio `name` — e só o `name`.

Bônus 2: descubra o que acontece se você trocar o `true` do `JWT.decode` por `false` e chamar `/me` com um token forjado. Depois desfaça.

Travou? `git checkout aula-03`.

Recapitulando

- Controller recebe e responde; a regra mora no service.
- Serializer é a lista de convidados do JSON — sem ele o digest vaza.
- O payload do JWT é público. Assinado ≠ criptografado.
- Logout de verdade precisa de denylist por `jti`; o TTL faz a limpeza sozinho.
- `token_version` derruba tudo de uma vez, e é o que a troca de senha usa.
- Em autenticação, respostas diferentes vazam informação. Uma resposta só, sempre.

Na próxima: nada disso serve se está só na sua máquina. Vamos colocar no ar.